

Spring Camp 2026

# 출렁이지 않는 편안함

Java AOT 컴파일과 **Project Leyden**

김민규

# 연사자 소개

10년차 백엔드 엔지니어

2024 – Robust serverless platform

- 애플리케이션 실행기

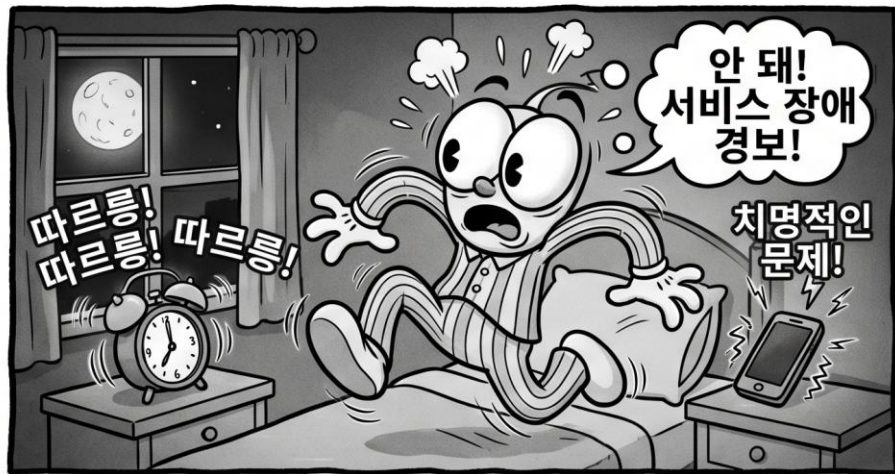
2016 – 회원플랫폼

- 네이버 2단계 인증 개선
- 잔여백신 예약
- 사내 인증 플랫폼 개발

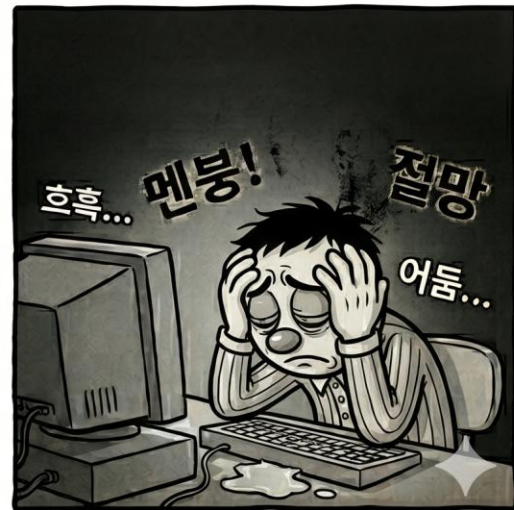


김민규

**스프링 부트 개발자는  
멘탈이 나가버렸습니다**







# 기나긴 대기



1. 트래픽 증가



2. 스케일 아웃 시도



3. 컨테이너 준비 지연



4. 기존 서버들의 연쇄 다운

# 늘어지는 레이턴시



1. 앱 실행 후 Cold 상태



2. Cold 상태에서  
메소드 Interpret



3. 요청 처리 지연으로  
인한 5xx 에러 발생



3. JIT 컴파일로 인한  
CPU 사용량 증가

# Java 3대 지연





# Project Leyden

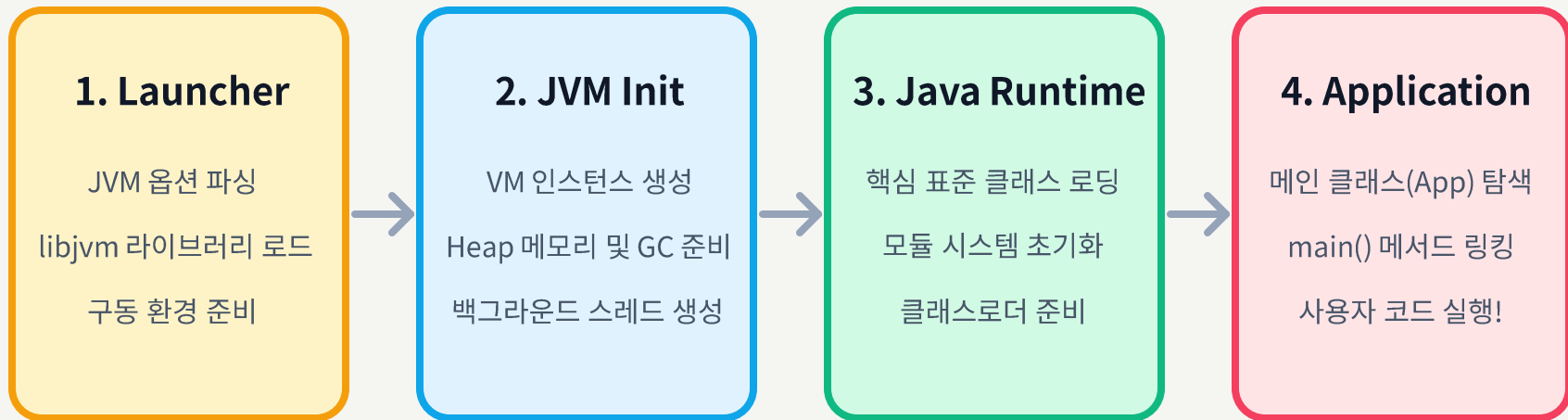
Startup time & Time to peek  
Less Footprint



# Bootstrap

# JVM 부트스트랩 핵심 요약

java 명령 실행부터 main() 메서드 호출까지





# Class Loading

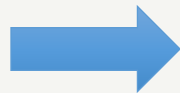
# Class Loading

```
A.java  
public class A {  
    public int foo() {  
        B b = new B();  
        return b.bar() + 1;  
    }  
}
```



# Class Loading

```
A.java
public class A {
    public int foo() {
        B b = new B();
        return b.bar() + 1;
    }
}
```



Bytecode

# Class Loading



Bytecode

## 1. Magic Number & Version

0xCAFEFABE, Minor & Major Version

## 2. Constant Pool (상수 풀) ★

심볼릭 레퍼런스 (텍스트 이름표) 보관소  
클래스명, 메서드명, 문자열 상수 등

## 3. Class Metadata & Fields

## 4. Methods (메서드) ★

실제 실행될 바이트코드 명령어 (Code Attribute)  
상수 풀의 인덱스를 참조하여 동작

## 5. Attributes

# Class Loading



Bytecode

## 2. Constant Pool (상수 풀) ★

심볼릭 레퍼런스 (텍스트 이름표) 보관소  
클래스명, 메서드명, 문자열 상수 등

# Class Loading



Bytecode

## 4. Methods (메서드) ★

실제 실행될 바이트코드 명령어 (Code Attribute)

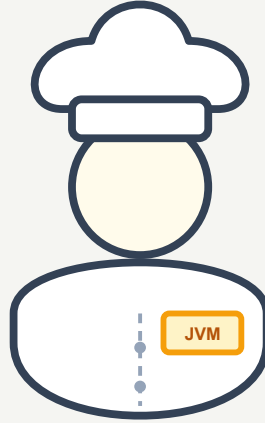
상수 풀의 인덱스를 참조하여 동작

# Class Loading



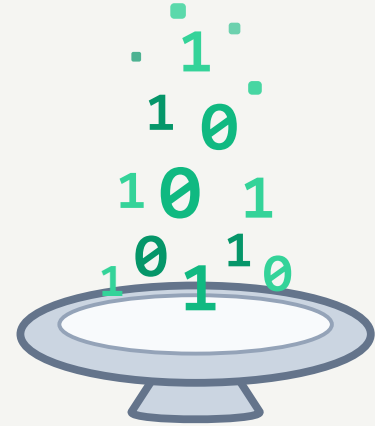
Bytecode

Read



JVM

Gen



Binary

EXAMPLE · TODAY'S CLASSES

## 오늘의 예제 — 클래스 A, B 그리고 main()

Main.java

```
void main() {  
    A a = new A();  
    System.out.println(a.foo());  
}
```

a.foo()

A.java

```
public class A {  
    public int foo() {  
        B b = new B();  
        return b.bar() + 1;  
    }  
}
```

b.bar()

B.java

```
class B {  
    public int bar() {  
        return 2;  
    }  
}
```

INTERPRETER · CONSTANT POOL RESOLUTION

## 0: new #7 — 상수풀에서 #7 찾기

A.class — public int foo()

```
▶ 0: new #7
  3: dup
  4: invokespecial #9
  7: astore_1
  8: aload_1
  9: invokevirtual #10
 12: iconst_1
 13: iadd
 14: ireturn
```

### Class A · Runtime Constant Pool

|     |                               |
|-----|-------------------------------|
| #7  | Class #8                      |
| #8  | Utf8 "B"                      |
| #9  | Methodref #8.#3               |
| #10 | Methodref #8.#11              |
| #11 | NameAndType #12:#13 "bar:()I" |
| #12 | Utf8 "bar"                    |
| #13 | Utf8 "()I"                    |



INTERPRETER · CONSTANT POOL RESOLUTION

## #7 = Class B — 아직 로딩되지 않았다

아직 로딩 전!

A.class — public int foo()

```
▶ 0: new #7
  3: dup
  4: invokespecial #9
  7: astore_1
  8: aload_1
  9: invokevirtual #10
 12: iconst_1
 13: iadd
 14: ireturn
```

### Class A · Runtime Constant Pool

|     |                               |
|-----|-------------------------------|
| #7  | Class #8 → Class B            |
| #8  | Utf8 "B"                      |
| #9  | Methodref #8.#3               |
| #10 | Methodref #8.#11              |
| #11 | NameAndType #12:#13 "bar:()I" |
| #12 | Utf8 "bar"                    |
| #13 | Utf8 "()I"                    |

DETOUR · CLASS LOADING — CLASS B

## Class B 로딩 — .class 파싱, Metaspace 할당

B.class bytecode

```
class B
```

```
constant_pool
```

```
Method <init>
```

```
Method bar():I
```

```
Code: iconst_2; ireturn
```

load & parse

### Metaspace · Method Area

Klass B metadata

@ 0x7000

Runtime Constant Pool

@ 0x8000

<init>():V

@ 0x9000

bar():I

@ 0x10000

DETOUR · CLASS LOADING — CLASS B

## Class B 검증 — 유효한 클래스인가?

B.class bytecode

```
class B
constant_pool
  Method <init>
  Method bar():I
Code: iconst_2; ireturn
```

verify

### Metaspace · Method Area

Klass B metadata



@ 0x7000

Runtime Constant Pool



@ 0x8000

&lt;init&gt;():V



@ 0x9000

bar():I



@ 0x10000

DETOUR · CLASS LOADING — CLASS B

## Class B 초기화 — static이 없으니 생략

Class B 로딩 완료

B.class bytecode

```
class B
constant_pool
  Method <init>
  Method bar():I
Code: iconst_2; ireturn
```

initialize

### Metaspace · Method Area

Klass B metadata



@ 0x7000

Runtime Constant Pool



@ 0x8000

&lt;init&gt;():V



@ 0x9000

bar():I



@ 0x10000

INTERPRETER · CONSTANT POOL RESOLUTION

## #7 → 0x7000 매핑 완료 — new 진행

A.class — public int foo()

```
▶ 0: new #7
  3: dup
  4: invokespecial #9
  7: astore_1
  8: aload_1
  9: invokevirtual #10
 12: iconst_1
 13: iadd
 14: ireturn
```

### Class A · Runtime Constant Pool

|     |                               |        |
|-----|-------------------------------|--------|
| #7  | Class #8 → Class B            | 0x7000 |
| #8  | Utf8 "B"                      |        |
| #9  | Methodref #8.#3 (init():V)    |        |
| #10 | Methodref #8.#11              |        |
| #11 | NameAndType #12:#13 "bar:()I" |        |
| #12 | Utf8 "bar"                    |        |
| #13 | Utf8 "()I"                    |        |

INTERPRETER · CONSTANT POOL RESOLUTION

## 4: invokespecial #9 — 생성자 호출

아직 링킹 전!

A.class — public int foo()

0: new #7

3: dup

▶ 4: invokespecial #9

7: astore\_1

8: aload\_1

9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

#10 Methodref #8.#11

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## <init> 링킹 — #9 → 0x9000

### Metaspace · Method Area

Klass B metadata



@ 0x7000

Runtime Constant Pool



@ 0x8000

&lt;init&gt;():V



@ 0x9000

bar():I



@ 0x10000

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref #8.#11

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## 링킹된 생성자 실행 — B 객체 생성

A.class — public int foo()

실행 ▶

0: new #7

3: dup

▶ 4: invokespecial #9

7: astore\_1

8: aload\_1

9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref #8.#11

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"



INTERPRETER · CONSTANT POOL RESOLUTION

## 7-8: astore\_1 · aload\_1 — 객체 사용 준비

A.class — public int foo()

0: new #7

3: dup

4: invokespecial #9

▶ 7: astore\_1

▶ 8: aload\_1

9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref #8.#11

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## 9: invokevirtual #10 — 일반 메소드 호출

A.class — public int foo()

0: new #7

3: dup

4: invokespecial #9

7: astore\_1

8: aload\_1

▶ 9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref #8.#11

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()"I"

INTERPRETER · CONSTANT POOL RESOLUTION

## 심볼 조합 — #10 = B.bar()I

A.class — public int foo()

0: new #7

3: dup

4: invokespecial #9

7: astore\_1

8: aload\_1

▶ 9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref B.bar()I

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## bar() 링킹 — #10 → 0x10000

### Metaspace · Method Area

Klass B metadata



@ 0x7000

Runtime Constant Pool



@ 0x8000

&lt;init&gt;():V



@ 0x9000

bar():I



@ 0x10000

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref B.bar():I

0x10000

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## 링킹된 bar() 실행

A.class — public int foo()

실행 ▶

0: new #7

3: dup

4: invokespecial #9

7: astore\_1

8: aload\_1

▶ 9: invokevirtual #10

12: iconst\_1

13: iadd

14: ireturn

### Class A · Runtime Constant Pool

#7 Class #8 → Class B

0x7000

#8 Utf8 "B"

#9 Methodref #8.#3 (init():V)

0x9000

#10 Methodref B.bar()I

0x10000

#11 NameAndType #12:#13 "bar:()I"

#12 Utf8 "bar"

#13 Utf8 "()I"

INTERPRETER · CONSTANT POOL RESOLUTION

## 12-14: iconst\_1 · iadd · ireturn — foo() 종료

A.class — public int foo()

```
0: new #7
3: dup
4: invokespecial #9
7: astore_1
8: aload_1
9: invokevirtual #10
```

▶ 12: iconst\_1

▶ 13: iadd

▶ 14: ireturn

### Class A · Runtime Constant Pool

|     |                               |         |
|-----|-------------------------------|---------|
| #7  | Class #8 → Class B            | 0x7000  |
| #8  | Utf8 "B"                      |         |
| #9  | Methodref #8.#3 (init():V)    | 0x9000  |
| #10 | Methodref B.bar()I            | 0x10000 |
| #11 | NameAndType #12:#13 "bar:()I" |         |
| #12 | Utf8 "bar"                    |         |
| #13 | Utf8 "()"I                    |         |



# JIT Compile

# JIT Compile

```
A.java
public class A {
    public int foo() {
        B b = new B();
        return b.bar() + 1;
    }
}
```



```
A.class - public int foo()
    0: new #7
    3: dup
    4: invokespecial #9
    7: astore_1
    8: aload_1
    9: invokevirtual #10
   12: iconst_1
   13: iadd
   14: ireturn
```



# JIT Compile

## C1 compiler

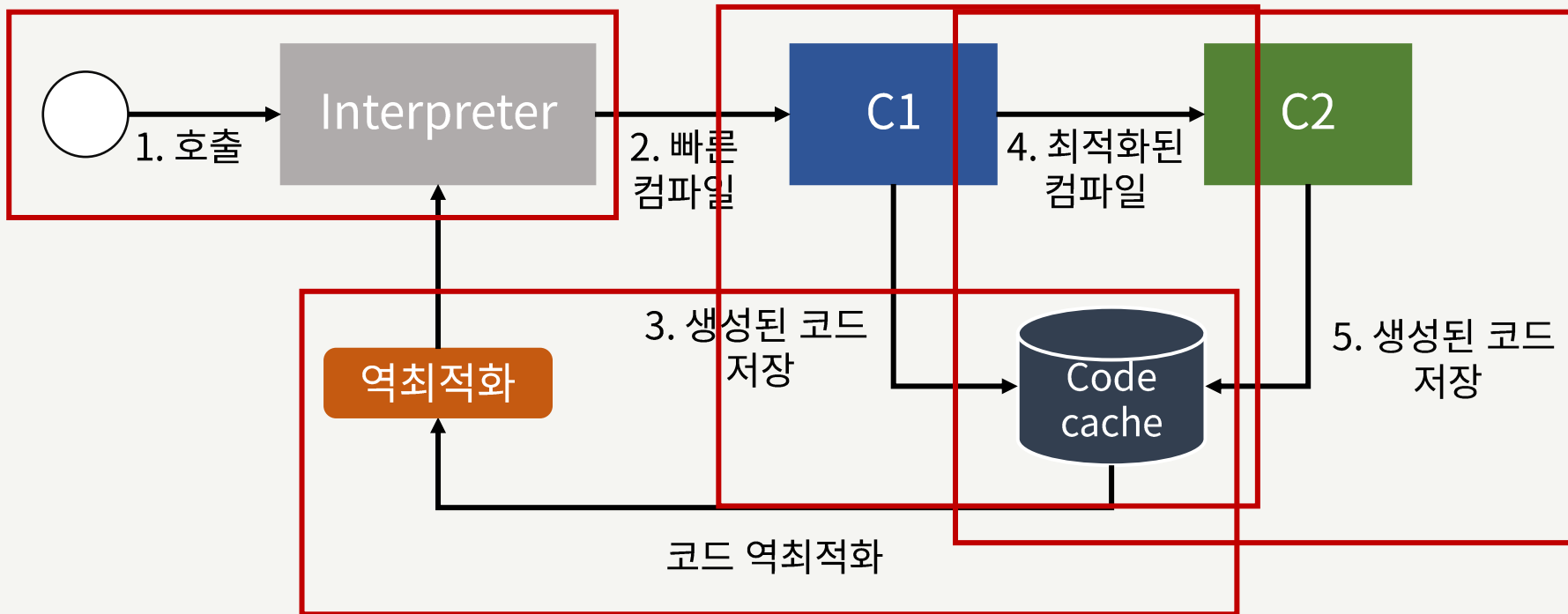
- 빠른 컴파일
- 빠른 실행
- 작은 애플리케이션
- 가벼운 최적화

## C2 compiler

- 빠른 속도
- 코드 실행 관찰 / 분석
- 서버 애플리케이션
- 공격적인 최적화

# JIT Compile

Tier?



# JIT Compile

## Tier?

| Tier 0      | Tier 1                      | Tier 2                  | Tier 3                       | Tier 4                     |
|-------------|-----------------------------|-------------------------|------------------------------|----------------------------|
| Interpreter | C1                          | C1                      | C1                           | C2                         |
| 바이트코드<br>해석 | 단순 기계어<br>컴파일<br>(프로파일링 없음) | 제한적<br>프로파일링<br>(호출 횟수) | 전체<br>프로파일링<br>(타입, 분기 패턴 등) | 최고 성능<br>최적화<br>(프로파일링 안함) |

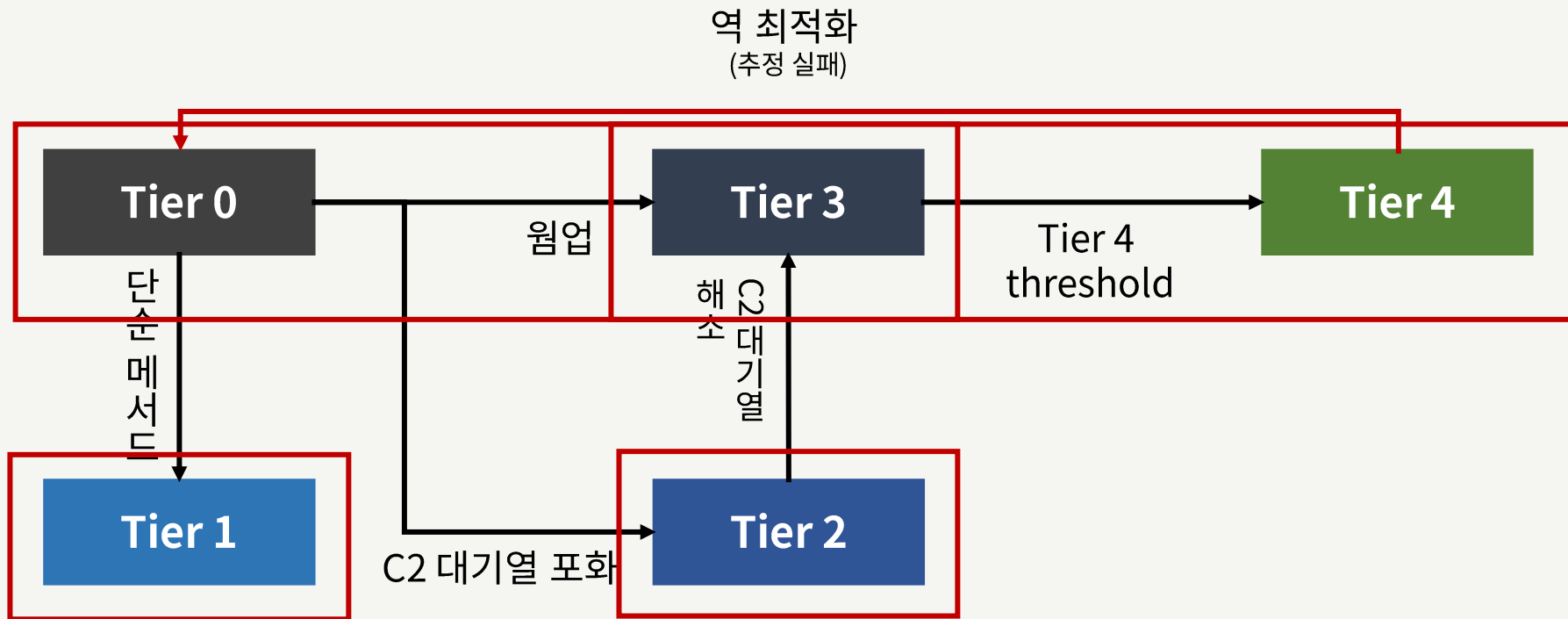
# JIT Compile

## Tier?

| Tier 0      | Tier 1                      | Tier 2                  | Tier 3                       | Tier 4                     |
|-------------|-----------------------------|-------------------------|------------------------------|----------------------------|
| Interpreter | C1                          | C1                      | C1                           | C2                         |
| 바이트코드<br>해석 | 단순 기계어<br>컴파일<br>(프로파일링 없음) | 제한적<br>프로파일링<br>(호출 횟수) | 전체<br>프로파일링<br>(타입, 분기 패턴 등) | 최고 성능<br>최적화<br>(프로파일링 안함) |

# JIT Compile

Tier?



# JIT Compile

JIT compile flag 확인: `-XX:+PrintFlagsFinal`

```
java -XX:+PrintFlagsFinal --version | grep CompileThreshold
    intx CompileThreshold                = 10000      {pd product} {default}
double CompileThresholdScaling          = 1.000000 {product} {default}
    uintx IncreaseFirstTierCompileThresholdAt = 50        {product} {default}
    intx Tier2CompileThreshold          = 0          {product} {default}
    intx Tier3CompileThreshold          = 2000       {product} {default}
    intx Tier4CompileThreshold          = 15000      {product} {default}
```

# Java 3대 지연



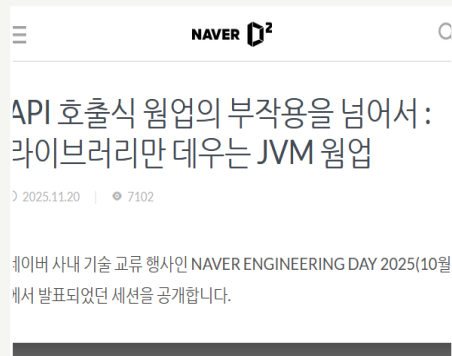
# 기존 해결 방식





# Warmup 스크립트

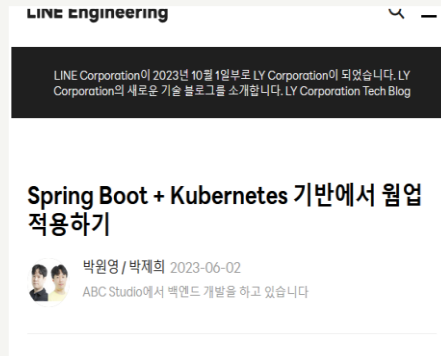
# Warmup 스크립트



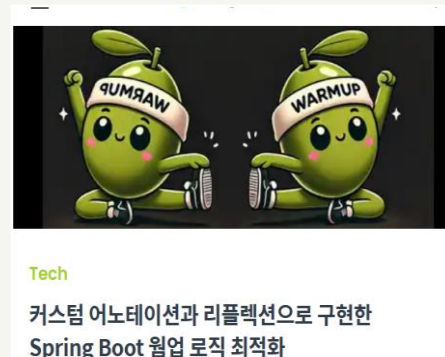
Naver



Kakao pay

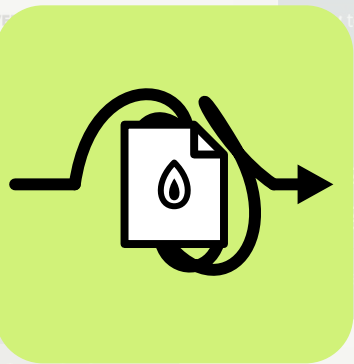


Line



올리브영

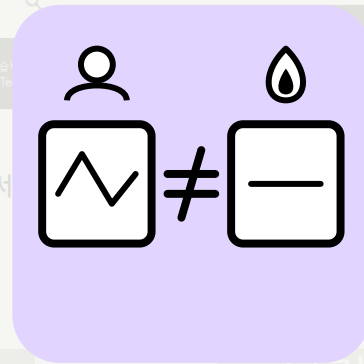
# Warmup 스크립트



복잡한 배포



전체 배포 시간 지연



최적 상태 재현 불가

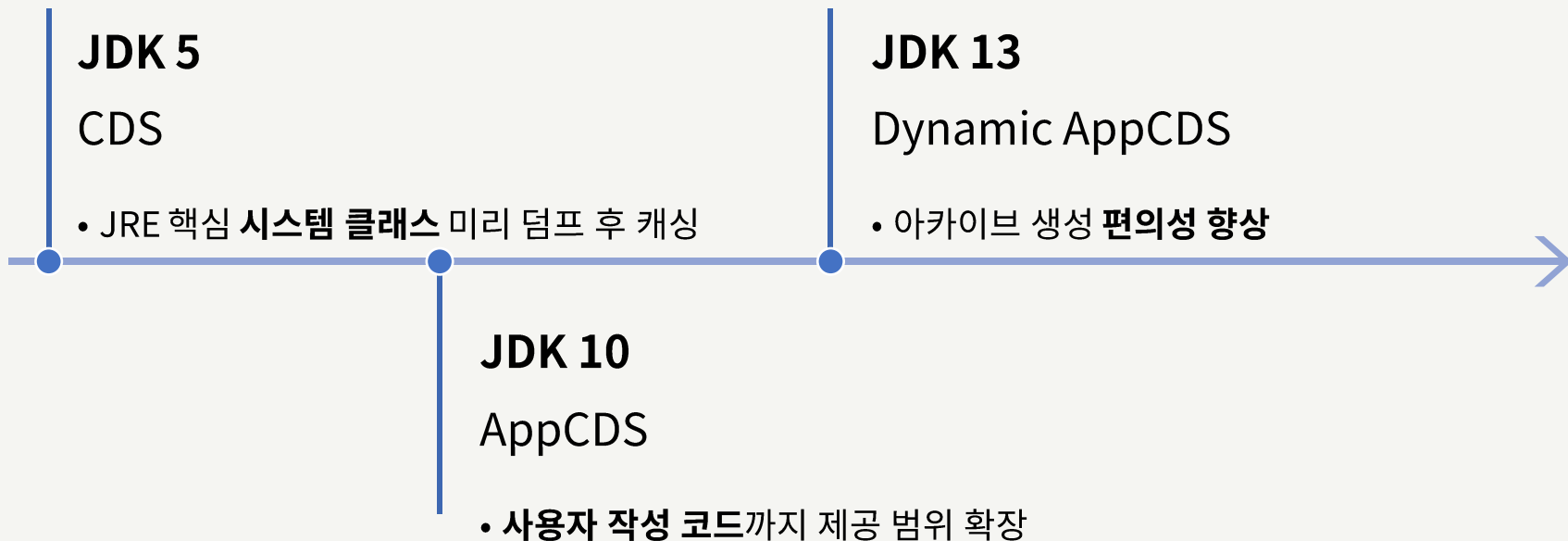
# **Class Data Sharing (CDS)**

# Class Data Sharing (CDS)

## CLASS LOADING PROCESS



# Class Data Sharing (CDS)



# Class Data Sharing (CDS)

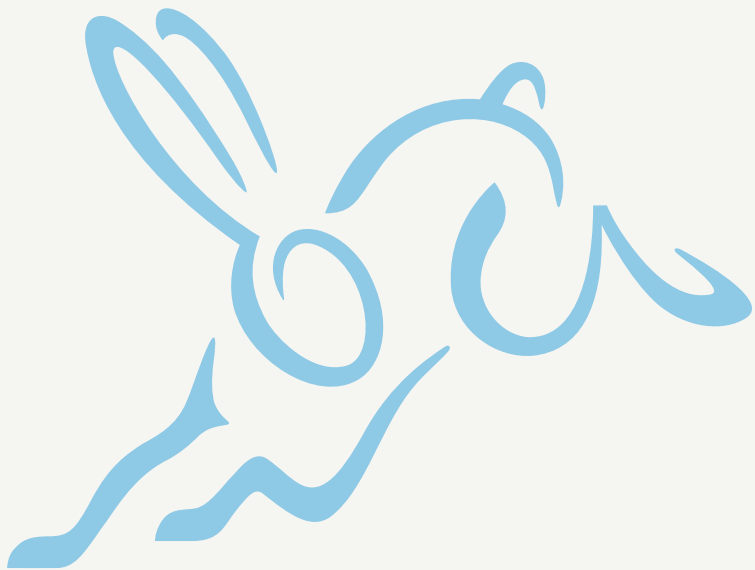


# GraalVM

## Native image



# GraalVM Native Image

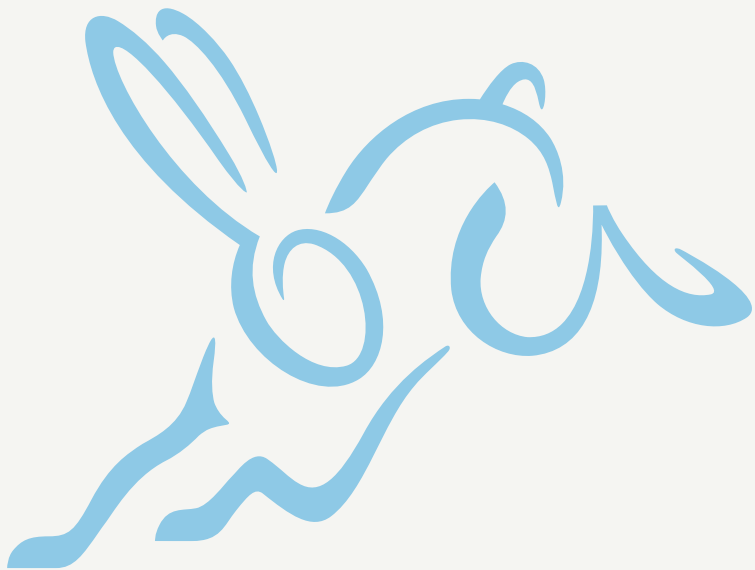


## JDK 애플리케이션의 Native화

기존 Java 애플리케이션의  
느린 **Bootstrap**, 높은 리소스 점유를  
해결하기 위해 만든 이미지

빌드 된 Java 애플리케이션을  
**AOT(Ahead-Of-Time)** 컴파일하여  
Native binary로 재 생성

# Native Image와 AOT



## JDK 애플리케이션의 Native화

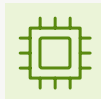
기존 Java 애플리케이션의  
**느린 Bootstrap, 높은 리소스 점유를**  
해결하기 위해 만든 이미지

빌드 된 Java 애플리케이션을  
**AOT(Ahead-Of-Time)** 컴파일하여  
Native binary로 재 생성

# Native Image



**빠른 시작 속도**



**적은 메모리**



**작은 컨테이너**

# Closed World Assumption

런타임에 필요한 모든 클래스, 메서드, 바이트코드와 그 의존성이  
빌드 시점에 **100% 확정**되어 있다

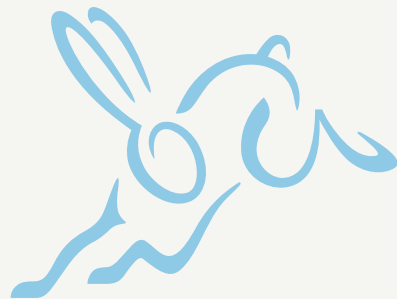
# Spring AOT

Spring Framework 내 **Reflection / dynamic proxy**를 정적 자바 코드 변환 & 런타임 Hint 생성

- Spring ApplicationContext를 빌드 타임에 분석해 애플리케이션 실행 환경을 확정
- 생성될 모든 빈을 확인, 리플렉션 대신 new를 통해 직접 객체를 생성하는 클래스 생성 & 런타임 힌트 제공



# Jar app VS Native Image



빌드 결과물

Jar

Executable Binary

실행 런타임

JVM

Substrate VM

Loading & Linking

Runtime

Ahead Of Time

내부 최적화

JIT Compile

PGO

# Native image 최적화

## 정적 코드 정보 (AOT 기본)

- 소스 코드
- 클래스/메서드 구조



**GraalVM  
AOT Compiler**



**최종 결과물**

**Native Image**

**빠른 시작 속도**

# Native image 최적화 - PGO

## 정적 코드 정보 (AOT 기본)

- 소스 코드
- 클래스/메서드 구조

## 동적 프로파일 (Runtime Data)

- 자주 호출되는 메서드 (Hot Code)
- 분기 예측 (Branch Prediction)
- 다형성 타입(Type) 추론 결과





# JIT vs AOT vs AOT + PGO

## Native Image

~65%

Peak Throughput

도달 시간      수 ms  
메모리 사용   매우 적음

## 일반 JVM (JIT)

100%

Peak Throughput

도달 시간      수 초 ~ 수 분  
메모리 사용      높음

## Native Image + PGO

~95%

Peak Throughput

도달 시간      수 ms  
메모리 사용   매우 적음

# GraalVM의 한계

자유로운 JVM vs 제한적인 GraalVM

## 동적 기능의 제약

Reflection / Proxy의  
**Runtime hint** 또는  
우회 코드 필요

## GC

G1 GC, Serial GC,  
Epsilon GC 만 지원  
저지연 GC 사용 불가

## Performance

JIT compile 대비  
퍼포먼스 고점 낮음



# Project Leyden

# Project Leyden의 목표



+



Memory  
Footprint

# Leyden Design Note: Shift and Constrain

Selectively **Shifting** and **Constraining**

## SHIFT

빌드 타임으로  
연산 이전

## CONSTRAIN

런타임 변경  
불가 제약

# Leyden Design Note: Condenser

응축기(Condenser)의 본격적인 도입

## Condenser (응축기) 란?

미리 수집한 클래스 로딩, 리플렉션 캐싱  
상태를 캐시파일로 응축

## AOT Cache (초기 부하 상쇄)

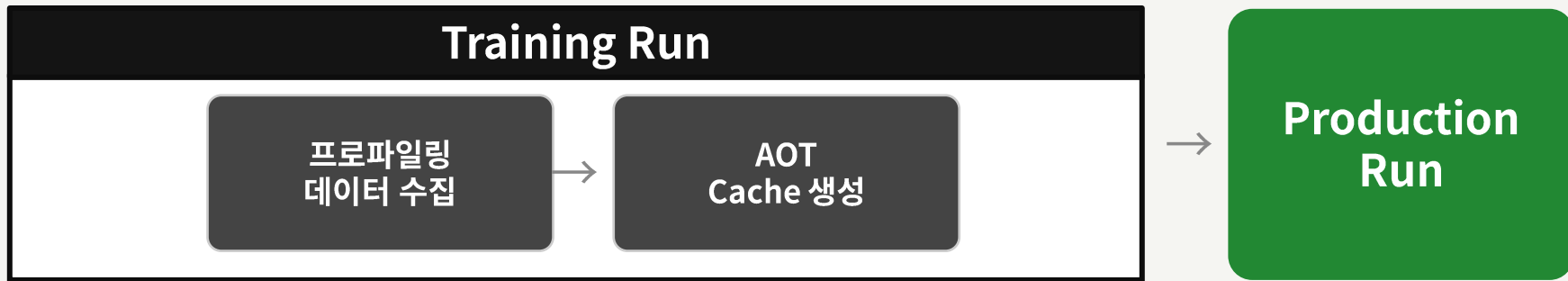
캐시를 즉시 JVM에 매핑하여  
링킹 및 부트스트랩 오버헤드 단축

기존의 AppCDS의 확장

# Leyden Design Note: Training Run

런타임 프로파일링 데이터를 기반으로 한 AOT 최적화 전략 고도화.

Training Run을 통해 실제 애플리케이션 실행 흐름을 분석 후 필요한 부분을 응축하여 최적화된 AOT Cache를 만들도록 결정



# AOTCache의 진화 흐름

## 캐시 범위

WIP

### AOT Code Compilation

기존 데이터 + JIT가 최적화한 네이티브 기계어 캐싱  
→ '최대(Peak)' 성능 도달 시간 최소화

JDK 25

### AOT Method Profile

기존 데이터 + JIT Compile 을 위한 Method profile  
→ JIT Compile 을 위한 월업 시간 감소

JDK24

### AOT Class loading

CDS + 검증 및 링크가 완료된 메타데이터 추가 누적 캐싱  
→ 로딩 & 링킹 오버헤드 제거로 '초기 기동(Startup)' 개선

### AppCDS (Base)

파싱(Parsed)된 클래스 메타데이터

## 사용성

JDK 26

Caching with  
Any GC

JDK 25

Command line 간소화



# AOTCache의 진화 흐름

## 캐시 범위

WIP

### AOT Code Compilation

기존 데이터 + JIT가 최적화한 네이티브 기계어 캐싱  
→ '최대(Peak)' 성능 도달 시간 최소화

JDK 25

### AOT Method Profile

기존 데이터 + JIT Compile 을 위한 Method profile  
→ JIT Compile 을 위한 월업 시간 감소

JDK24

### AOT Class loading

CDS + 검증 및 링크가 완료된 메타데이터 추가 누적 캐싱  
→ 로딩 & 링킹 오버헤드 제거로 '초기 기동(Startup)' 개선

### AppCDS (Base)

파싱(Parsed)된 클래스 메타데이터

## 사용성

JDK 26

Caching with  
Any GC

JDK 25

Command line 간소화

# AOTCache의 진화 흐름

## 캐시 범위

WIP

### AOT Code Compilation

기존 데이터 + JIT가 최적화한 네이티브 기계어 캐싱  
→ '최대(Peak)' 성능 도달 시간 최소화

JDK 25

### AOT Method Profile

기존 데이터 + JIT Compile 을 위한 Method profile  
→ JIT Compile 을 위한 월업 시간 감소

JDK24

### AOT Class loading

CDS + 검증 및 링크가 완료된 메타데이터 추가 누적 캐싱  
→ 로딩 & 링킹 오버헤드 제거로 '초기 기동(Startup)' 개선

### AppCDS (Base)

파싱(Parsed)된 클래스 메타데이터

## 사용성

JDK 26

Caching with  
Any GC

JDK 25

Command line 간소화

# AOTCache의 진화 흐름

캐시 범위

사용성

WIP

## AOT Code Compilation

기존 데이터 + JIT가 최적화한 네이티브 기계어 캐싱  
→ '최대(Peak)' 성능 도달 시간 최소화

JDK 25

## AOT Method Profile

기존 데이터 + JIT Compile 을 위한 Method profile  
→ JIT Compile 을 위한 월업 시간 감소

JDK24

## AOT Class loading

CDS + 검증 및 링크가 완료된 메타데이터 추가 누적 캐싱  
→ 로딩 & 링킹 오버헤드 제거로 '초기 기동(Startup)' 개선

## AppCDS (Base)

파싱(Parsed)된 클래스 메타데이터

기존 JIT compile tier 정책은 동일  
ng with GC

JDK 25

Command line 간소화

# Project Leyden: AOT Cache 생성 과정

1

2

3

4

5

6

7

8

RECORD & PREIMAGE

CREATE · ASSEMBLY

PHASE 1 · RECORD & PREIMAGE — TRAINING RUN

## ① Training Run 모드 활성화



```
$ java \  
  -XX:AOTCacheOutput=app.aot \  
  -jar app.jar
```

### JVM 시작

✓ -XX:AOTCacheOutput 플래그 감지

**Training Run 모드 ON**

이번 실행은 AOT cache 생성을 위한  
훈련(training) 실행임을 인지

JVM 동작은 평소와 동일 — “기록할 준비”를 갖추고 애플리케이션 시작

# Project Leyden: AOT Cache 생성 과정

1 2 3 4 5 6 7 8  
RECORD & PREIMAGE CREATE · ASSEMBLY

PHASE 1 · RECORD & PREIMAGE — TRAINING RUN

## ② 애플리케이션 실행 — 관찰과 프로파일링

추가 동작 없음 · 관찰만

### Training Run JVM — 평소와 동일

JVM 부트스트랩



클래스 로딩 · 링킹



애플리케이션 실행 main()

관찰 (프로파일링)

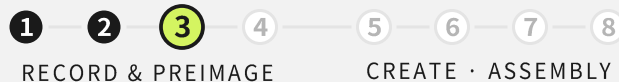
### Metaspace

클래스 로딩 메타데이터

상수 풀(Constant Pool) 상태

메소드 실행 프로파일

# Project Leyden: AOT Cache 생성 과정



PHASE 1 · RECORD & PREIMAGE — TRAINING RUN

## ③ 종료 시 Preimage Dump

애플리케이션 종료

**process exit**

실행 중 쌓아 둔 training data를  
종료 시점에 모아서 기록

Training Run JVM

Preimage dump

**app.aot.preimage**

Training run 동안 수집한 데이터를  
담은 중간 산출물

## PHASE 1 · RECORD & PREIMAGE — TRAINING RUN

### ④ Preimage — Create 단계를 위한 작업 지시서

#### app.aot.preimage

##### ① 클래스 메타데이터

training run에서 로드된 클래스 정보

##### ② 메소드 프로파일링 데이터

어떤 메소드가 어떻게 실행되었는지

##### ③ Create 단계가 재현할 작업 목록

클래스 로드 · 링크 · resolve 작업 지시서

#### 아직 최종 캐시가 아니다

create 단계가 그대로 따라 할 수 있는  
“작업 지시서”와 그 재료를 담은 중간 산출물



다음: Create 단계의 입력

PHASE 2 · CREATE — ASSEMBLY JVM

## ⑤ Assembly JVM 실행 — Preimage 검증

### Training JVM

preimage 기록 후 종료 단계

부모 프로세스

spawn

**app.aot.preimage**  
검증 대상 입력

### Assembly JVM (자식 JVM)

#### CDSConfig 검사

AOT cache 생성 설정이 유효한지 확인



#### preimage magic number 검증

올바른 preimage 파일인지 확인



#### preimage version 검증

JVM 버전과 호환되는지 확인





PHASE 2 · CREATE — ASSEMBLY JVM

## ⑥ Preimage 재생 — 로드·링크 상태 재구성

### Preimage 작업 목록

클래스 로드

메소드 링크

Resolved 상수 풀

재생 (replay)

### Assembly JVM — 재구성된 상태

Loaded Classes



Linked Methods



Resolved Constant Pool



# Project Leyden: AOT Cache 생성 과정

1 2 3 4 5 6 7 8  
RECORD & PREIMAGE CREATE · ASSEMBLY

PHASE 2 · CREATE — ASSEMBLY JVM

## ⑦ AOT Code Cache 생성

핵심 차별점: 머신 코드까지 미리 준비



**Training Data**

메소드 실행 프로파일

**JIT 컴파일러**

프로파일을 replay 하여  
필요한 메소드를 컴파일

**컴파일된 코드**

미리 만들어진 머신 코드

**Archive Code  
Region**

# Project Leyden: AOT Cache 생성 과정

1

2

3

4

5

6

7

8

RECORD & PREIMAGE

CREATE · ASSEMBLY

PHASE 2 · CREATE — ASSEMBLY JVM

## ⑧ 최종 조립 — app.aot 작성

### Archive Writer

재구성 된 클래스

컴파일 된

write

### app.aot

header

metadata (클래스 · 메소드)

training data

archived heap

AOT code

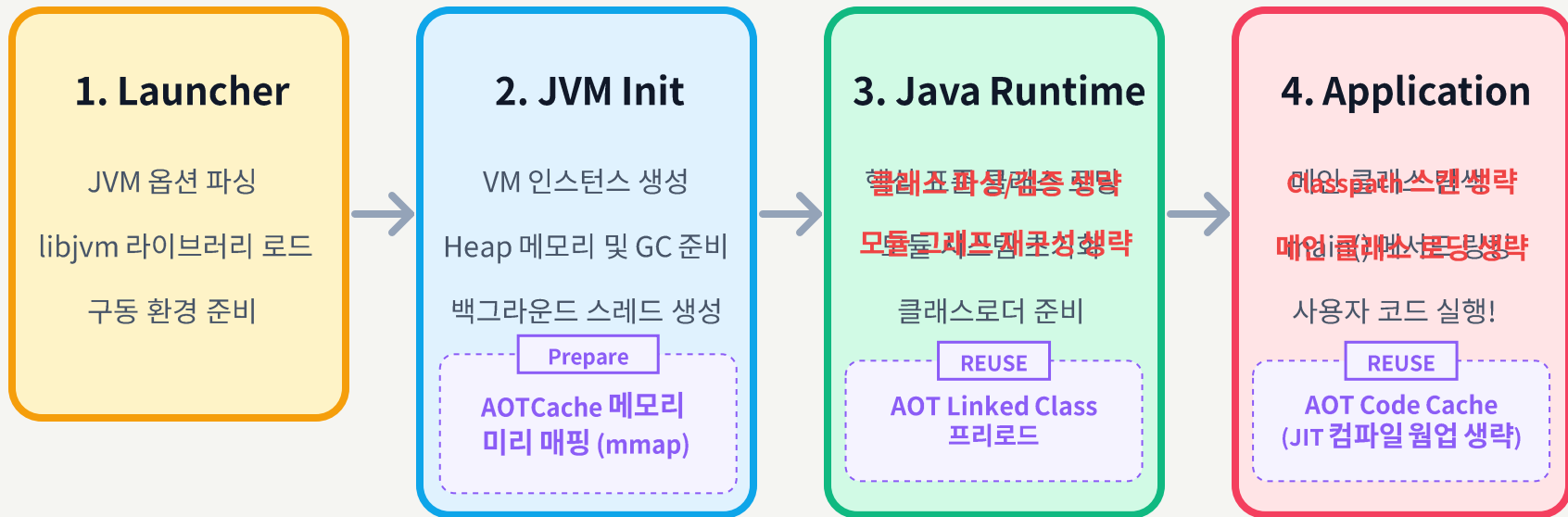
bitmap

✓ 완성

다음 실행부터는  
이 캐시를 읽어  
부트스트랩을 건너뛰다

# JVM 부트스트랩 AOTCache 적용

## java 명령 실행부터 main() 메서드 호출까지



# JVM 부트스트랩 AOTCache 적용

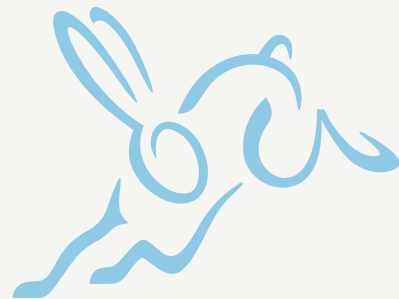
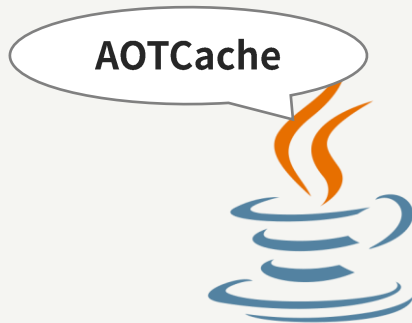
A.class - public int foo()

```
▶ 0: new #7
  3: dup
  4: invokespecial #9
  7: astore_1
  8: aload_1
  9: invokevirtual #10
 12: iconst_1
 13: iadd
 14: ireturn
```

## Class A · Runtime Constant Pool

|     |                               |         |
|-----|-------------------------------|---------|
| #7  | Class #8 → Class B            | 0x7000  |
| #8  | Utf8 "B"                      |         |
| #9  | Methodref #8.#3 (init():V)    | 0x9000  |
| #10 | Methodref B.bar()I            | 0x10000 |
| #11 | NameAndType #12:#13 "bar:()I" |         |
| #12 | Utf8 "bar"                    |         |
| #13 | Utf8 "()"I"                   |         |

# Project Leyden VS Native Image



**빌드 결과물**

Jar

Executable Binary

**실행 런타임**

JVM

Substrate VM

**Loading & Linking**

**Ahead Of Time**

**Ahead Of Time**

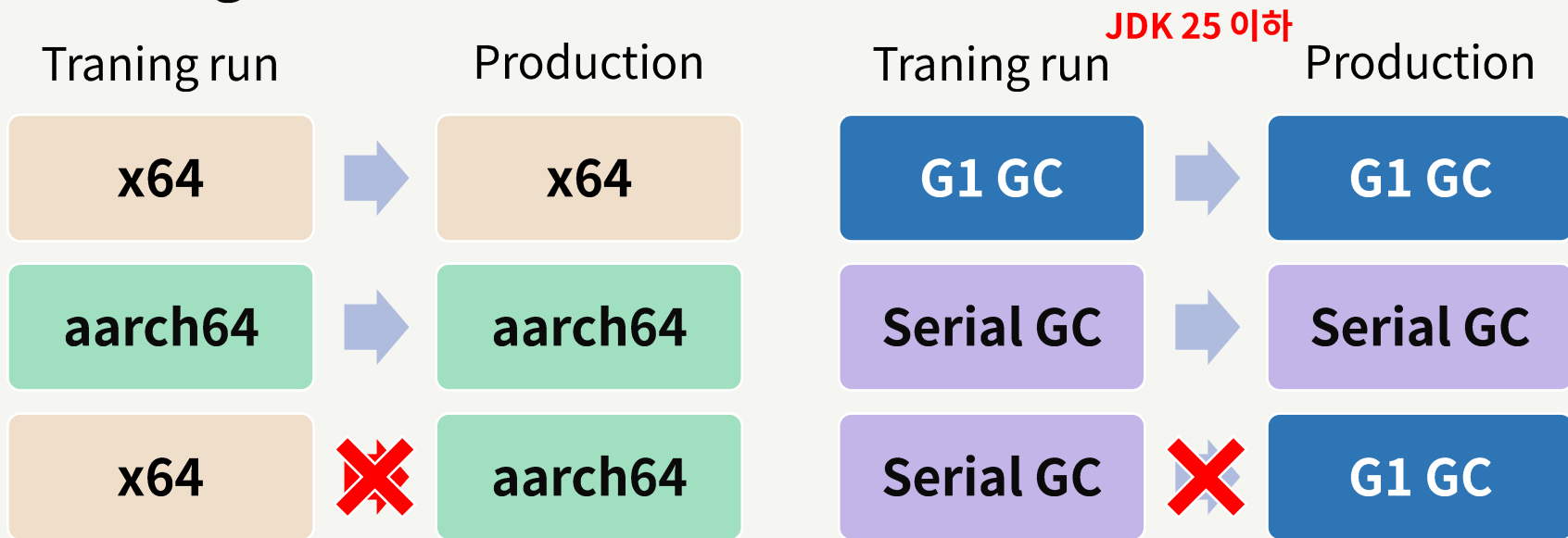
**내부 최적화**

**JIT Compile**

**PGO**

# AOTCache 주의사항

## Training run / Production 환경 동기화



# 실제 실행 전략



# AOT Cache 적용

## STEP 1

### 추출 및 캐시 생성 (Training Run)

JVM이 애플리케이션을 실행하면서 클래스 로딩·링크 데이터를 캐시 파일로 저장

```
$ java \  
  -XX:AOTCacheOutput=app.aot \  
  -Dspring.context.exit=onRefresh \  
  -jar app.jar
```

#### -XX:AOTCacheOutput

AOT Cache 생성 경로  
Training run 후 캐시 생성

#### -Dspring.context.exit =onRefresh

Spring은 Bean 초기화 후  
프로세스 종료

# AOT Cache 적용

## STEP 2

### 캐시를 적용하여 실행 (Production Run)

생성된 .aot 캐시를 JVM에 주입하여 클래스 로딩·링킹 오버헤드 제거

```
$ java \  
  -XX:AOTCache=app.aot \  
  -jar app.jar
```

### -XX:AOTCache

AOT Cache 활성화  
Training Run의 캐시 파일 사용

# AOT Cache 적용 결과

## AOT CACHE SIZE

생성된 캐시 파일 크기



# 115 MB

AOT Cache 파일 단일 크기

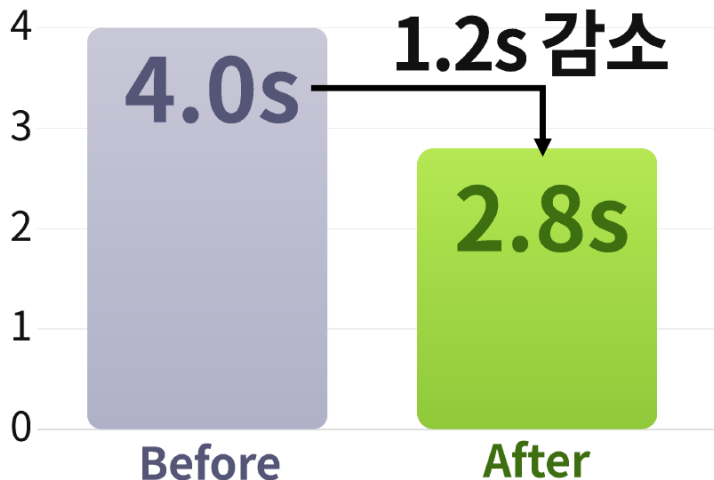


비교 참고

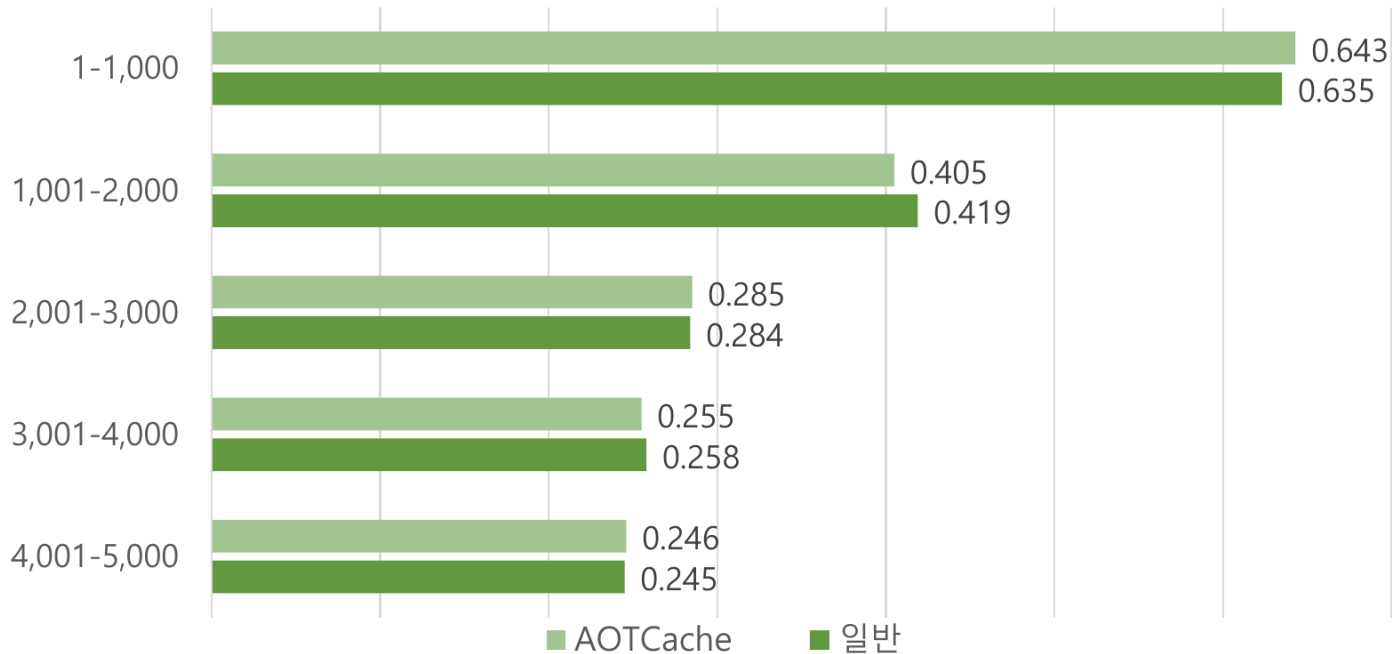
Spring PetClinic JAR  $\approx$  70 MB

## STARTUP TIME

애플리케이션 기동 시간

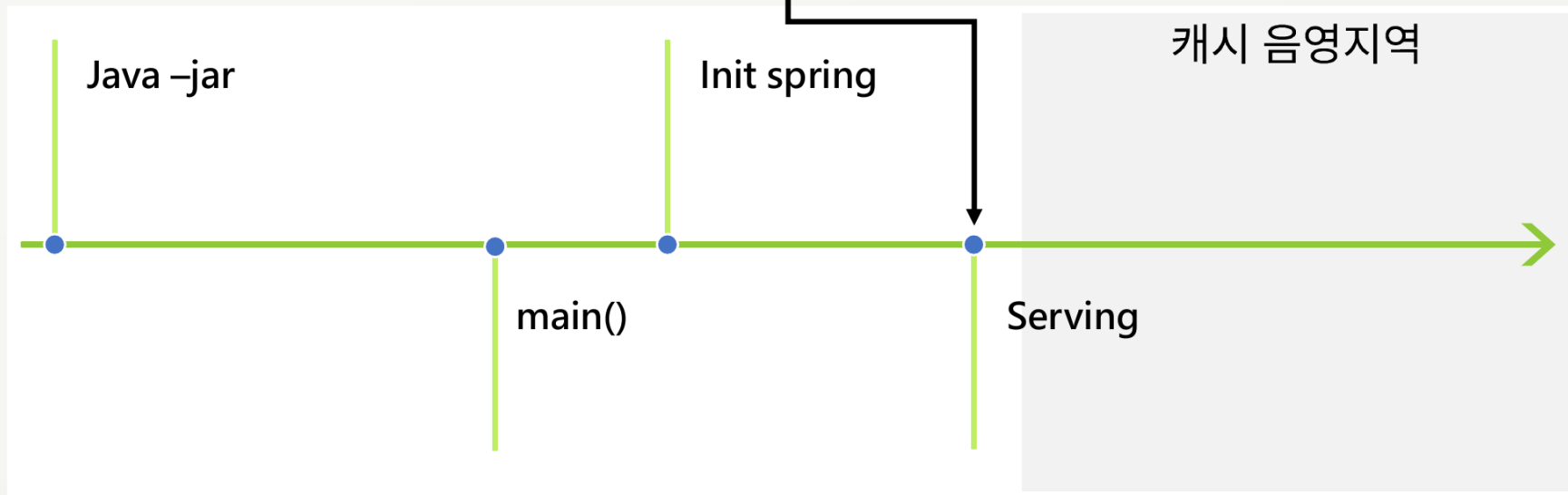


# 구간별 중앙값 레이턴시 비교



# 캐싱 범위

`-Dspring.context.exit=onRefresh`



# AOT Cache 생성 전략



# **Project Leyden w/ Spring petclinic**

# 애플리케이션 실행 타입

## Cache 없음

캐시 미사용

API Serving

Spring Bootstrap

JVM & Infra Boot

## Bootstrap

컨텍스트 초기화 까지 캐시

API Serving

Spring Bootstrap

JVM & Infra Boot

## Serving

실제 API 서빙까지 캐시

API Serving

Spring Bootstrap

JVM & Infra Boot



# 애플리케이션 빌드 - 웹업 방식

```
@Profile("caching")  
@Configuration(proxyBeanMethods = false)  
class ApiStartupCaller {
```

# 애플리케이션 빌드 - 웹업 방식

```
@EventListener(ApplicationReadyEvent.class)
void onApplicationReady() {
    // ...
    for (int i = 0; i < REPEAT_COUNT; i++) {
        for (String endpoint : TARGET_ENDPOINTS) {
            try {
                client.get().uri(endpoint).retrieve().toBodilessEntity();
            }
            catch (Exception ex) {
                // error handling
            }
        }
    }
    this.context.close();
}
```

# 애플리케이션 빌드 – Docker image

```
FROM eclipse-temurin:17-jdk AS builder
WORKDIR /app

COPY gradle/ gradle/
COPY gradlew build.gradle settings.gradle ./
RUN chmod +x gradlew

RUN ./gradlew dependencies --no-daemon

COPY src/ src/
RUN ./gradlew bootJar --no-daemon
RUN mv build/libs/*.jar build/libs/app.jar
```

# 애플리케이션 빌드 - 실행 타입

## Cache 없음

캐시 미사용

## Bootstrap

컨텍스트 초기화 까지 캐시

## Serving

실제 API 서빙까지 캐시

```
FROM eclipse-temurin:25-jdk AS run-default
WORKDIR /app
COPY --from=builder /app/build/libs/app.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

# 애플리케이션 빌드 - 실행 타입

Cache 없음

캐시 미사용

Bootstrap

컨텍스트 초기화 까지 캐시

Serving

실제 API 서빙까지 캐시

# COPY는 기존과 동일

```
RUN java -XX:AOTCacheOutput=app.aot \  
-Dspring.context.exit=onRefresh -jar app.jar
```

```
ENTRYPOINT ["java", "-XX:AOTCache=app.aot", "-jar", "app.jar"]
```

# 애플리케이션 빌드 - 실행 타입

## Cache 없음

캐시 미사용

---

## Bootstrap

컨텍스트 초기화 까지 캐시

---

## Serving

실제 API 서빙까지 캐시

---

# COPY는 기존과 동일

```
RUN java -XX:AOTCacheOutput=app.aot \  
-jar app.jar --spring.profiles.active=caching
```

```
ENTRYPOINT ["java", "-XX:AOTCache=app.aot", "-jar", "app.jar"]
```

# 테스트 시나리오

## 1. 초기 지표 수집

컨테이너 시작



시작 시간



메모리 사용량

## 2. API 호출 & 레이턴시 수집

15,000회 요청



레이턴시

## 3. 결과 Summary

구간별 지표 산출



레이턴시 종합



CPU / GC

# 케이스 별 비교



# 테스트 환경

JDK: Temurin 17(build)/25(Run)

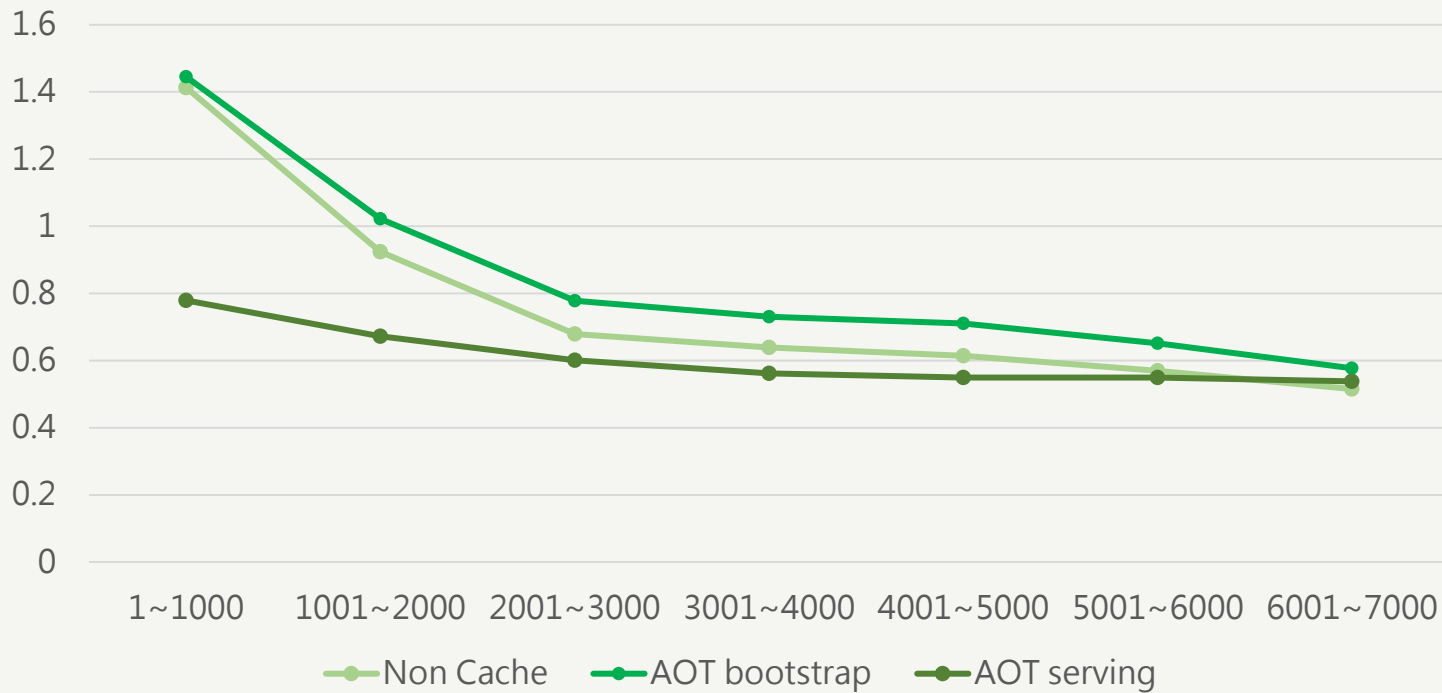
OS: Debian 13

Kernel: Linux 7.0.10

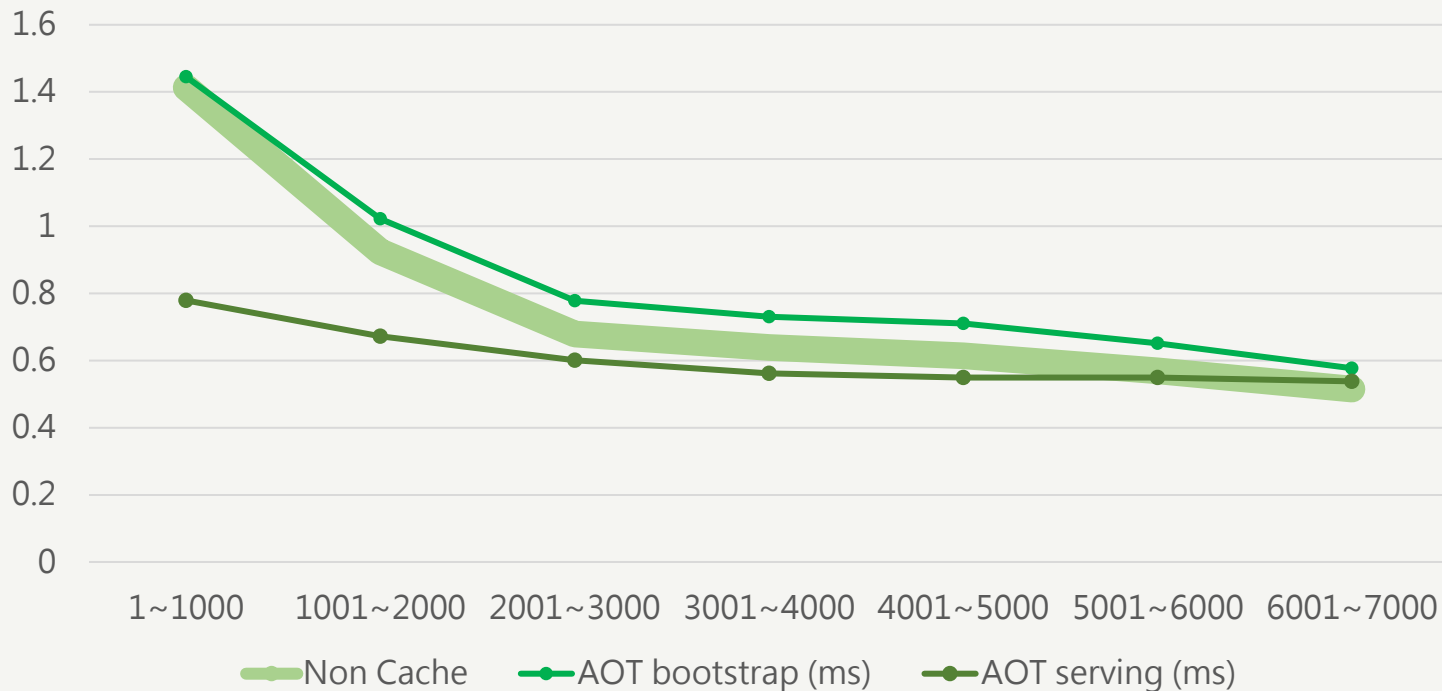
CPU: AMD Ryzen 5 9600X 6C 12T

Memory: 32 GiB

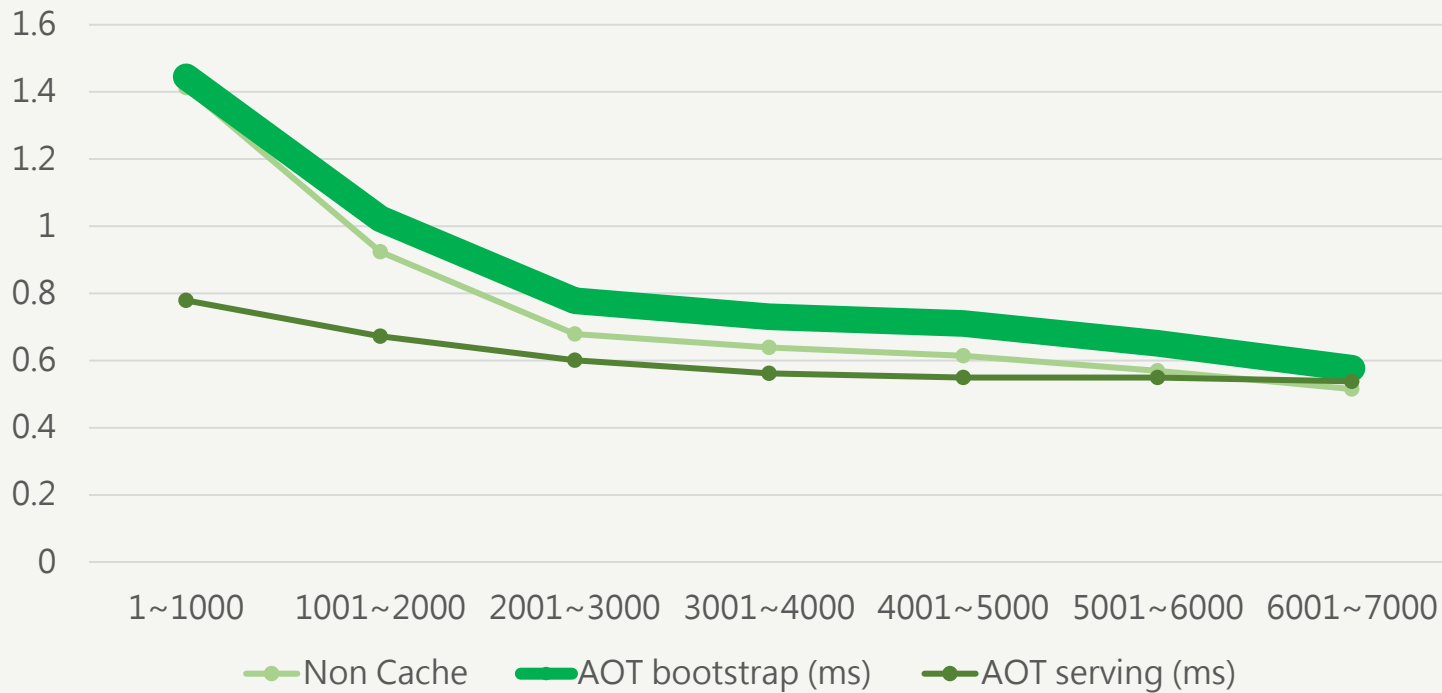
# 레이턴시 비교 - 중위



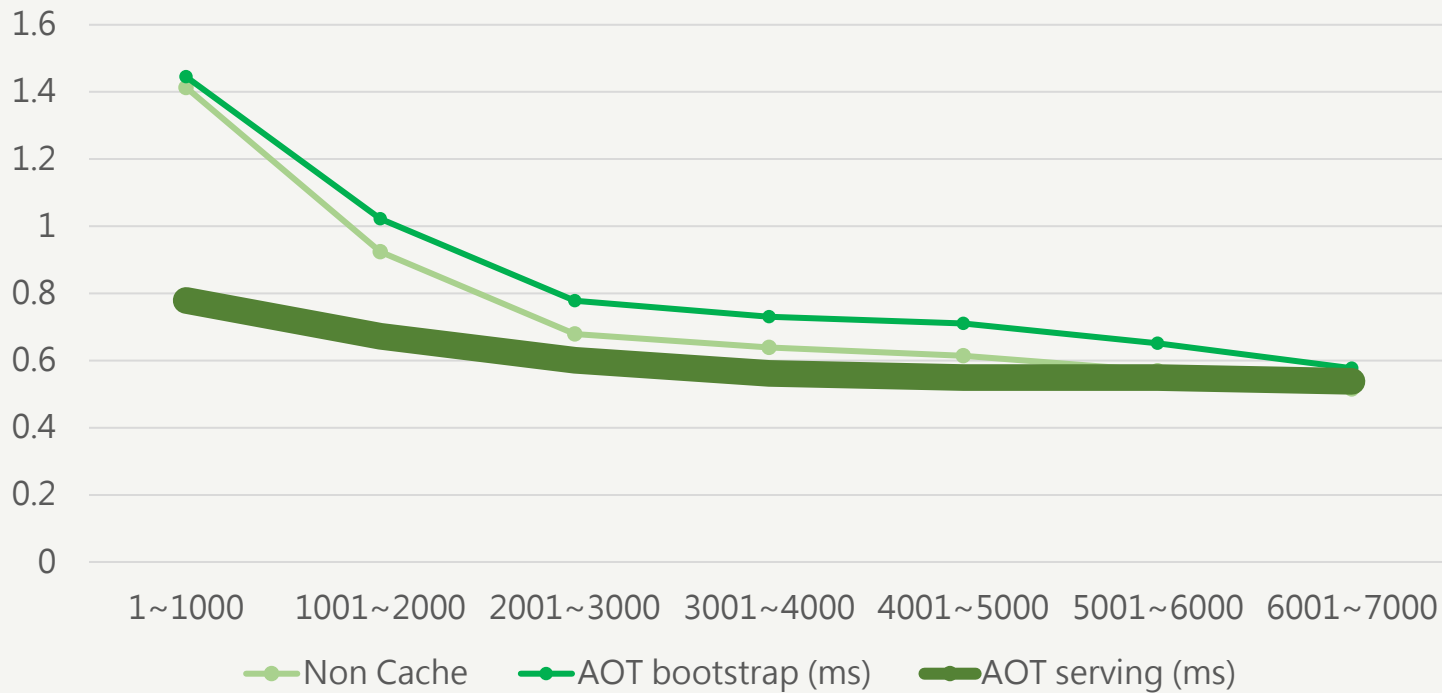
# 레이턴시 비교 - 중위



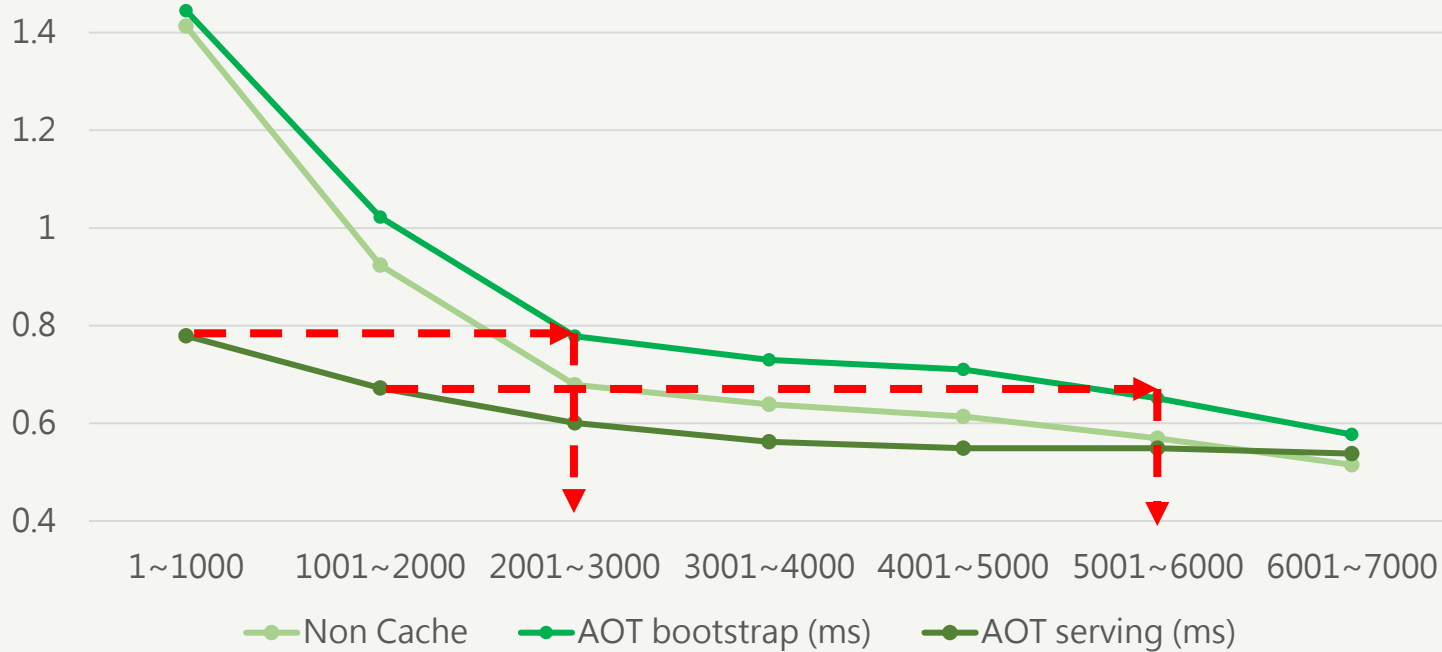
# 레이턴시 비교 - 중위



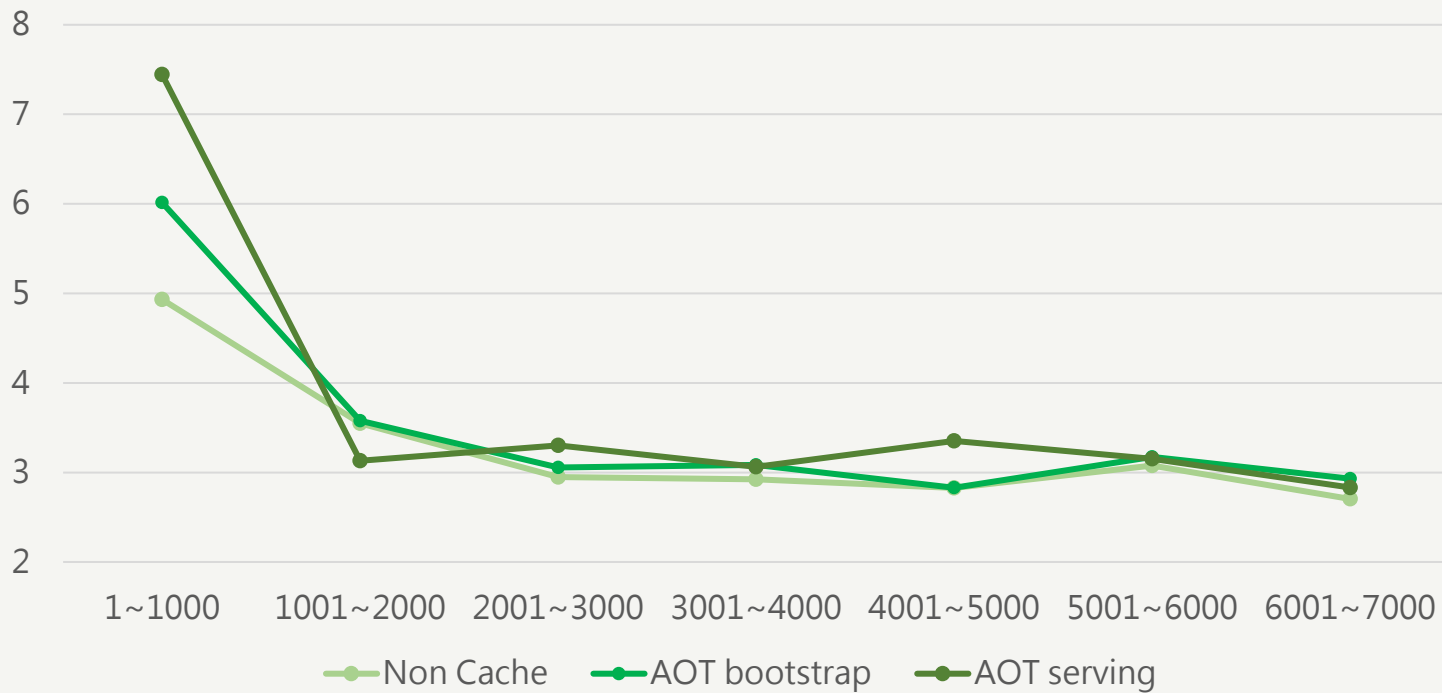
# 레이턴시 비교 - 중위



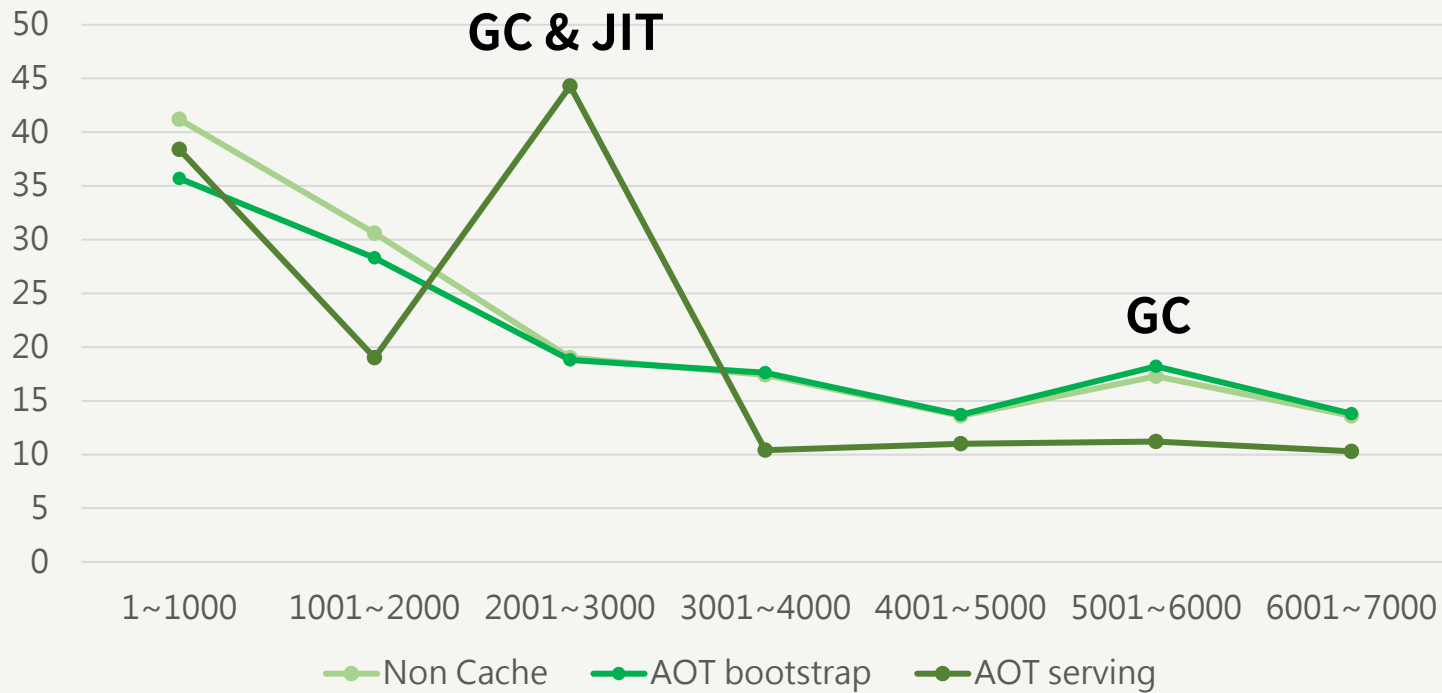
# 레이턴시 비교 - 중위



# 레이턴시 비교 – P99.9



# CPU 사용량



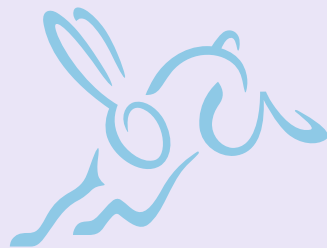


# AOT vs JIT



**JIT Compile**

런타임 최적화



**AOT Build**

빠른 초기 속도

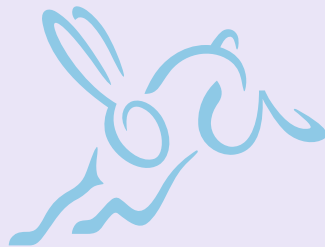
# AOT & JIT

## Project Leyden



### JIT Compile

런타임 최적화



### AOT Build

빠른 초기 속도







# 감사합니다

테스트 예제

<https://github.com/ksmail13/spring-petclinic>



문의 [ksmail13@gmail.com](mailto:ksmail13@gmail.com)